

Lecture 06: Interrupts

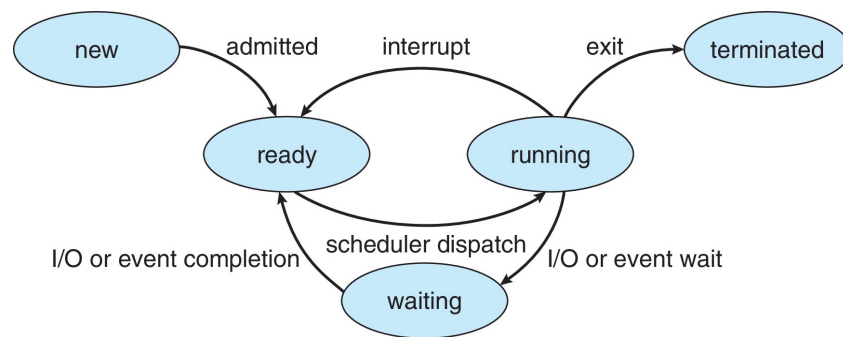
Vivek Kumar

Computer Science and Engineering

IIIT Delhi

vivekk@iiitd.ac.in

Last Lecture



Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	Process ₀ initiates I/O
4	Blocked	Running	Process ₀ is blocked, so Process ₁ runs
5	Blocked	Running	
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	Process ₁ now done
9	Running	–	
10	Running	–	Process ₀ now done

Figure 4.4: Tracing Process State: CPU and I/O

```

struct task_struct {
    unsigned int          _state;
    void                  *stack;
    int                    prio;
    unsigned int           _prio;
    int                    nr_cpus_allowed;
    struct mm_struct       *mm;
    struct mm_struct       *active_mm;

    pid_t                  pid;
    struct list_head       children;
    struct list_head       sibling;
    struct task_struct     *group_leader;

    /*
     * executable name, excluding path.
     *
     * - normally initialized setup_new_exec()
     * - access it with [gs]et_task_comm()
     * - lock it with task_lock()
     */
    char                    comm[TASK_COMM_LEN];

    struct nameidata        *nameidata;
    /* Filesystem information: */
    struct fs_struct        *fs;

    /* Open file information: */
    struct files_struct     *files;

```

- Process is a program in execution
- Processes have a life cycle
- OS has a datastructure (PCB) to store the details of a process

Today's Class

- Unix architecture
- System calls
- **Quiz-1**

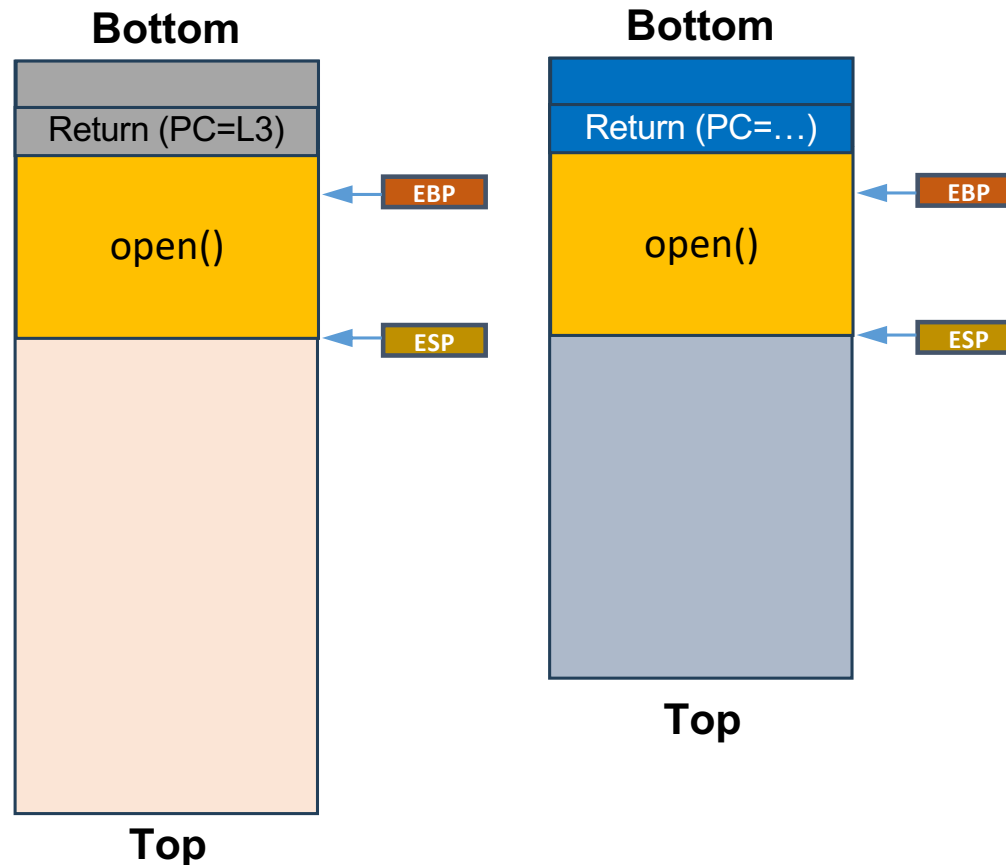
The OS Story Evolving So Far...

1. A **cook** (OS) manages a **kitchen** (*computer*)
2. The job of a cook is to do **cooking** (*execute binaries*) by reading **recipes** (*executable file*) in a **standard format** (*ELF*)
3. The cook has received recipe of a dish in **different files** (*relocatable files*). It will have to use a **stapler** (*linker*) to stitch these files to make a final recipe
4. The cook has to **store** (*load*) the **steps** (*instructions*) from the recipe in his memory to start cooking. The cook then execute instructions **one by one** (*call stack*) for cooking
5. The kitchen has gained a lot of popularity and have started receiving lot of recipes for cooking. The cook has hired several **helpers** (*processes*), where one helper cooks one recipe
6. To ensure the helpers don't fight for kitchen resources and make a mess, the ground rule is set that all helpers must talk to the head cook via a **hotline** (*interrupts*) if they need to access any kitchen resources

Motivation: Return Address on Stack can be Modified

```

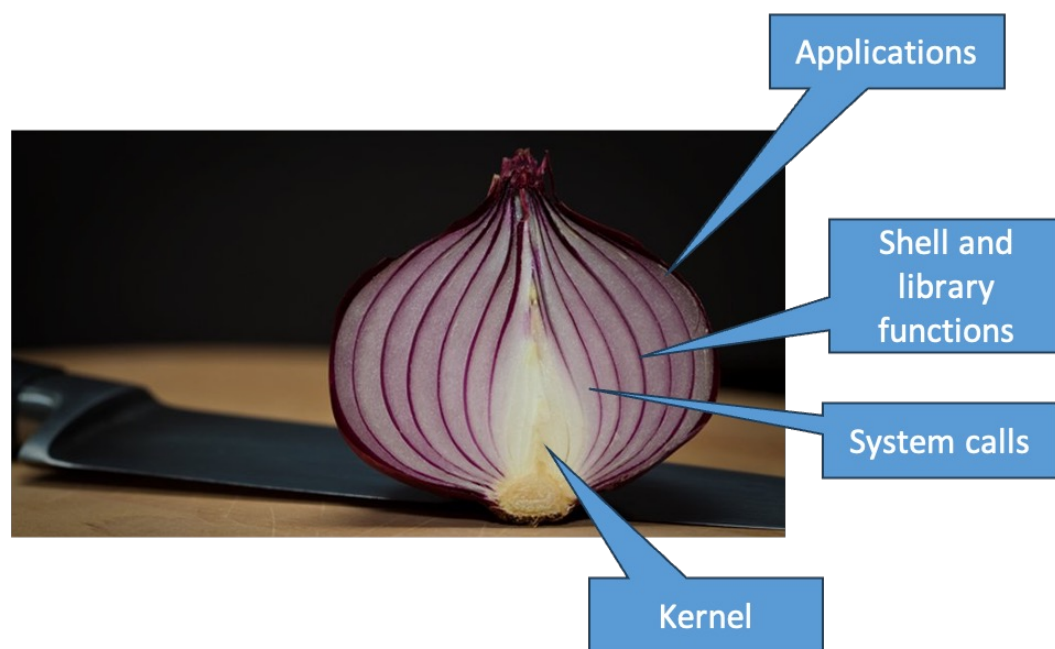
L1: main () {
    <save_regs>
    <save(PC)>
L2:  open();
    <restore(PC)>
    <restore_regs>
L3: }
  
```



● Issues

1. If **open()** is a regular method, then it's machine code won't be able to have a foolproof check if the calling process has the permission to access this file
 - If the process is compromised, an attacker can overwrite that return address on the stack
2. The main() called **open()**, but then **open** could be **forced** to return to some malicious process call stack

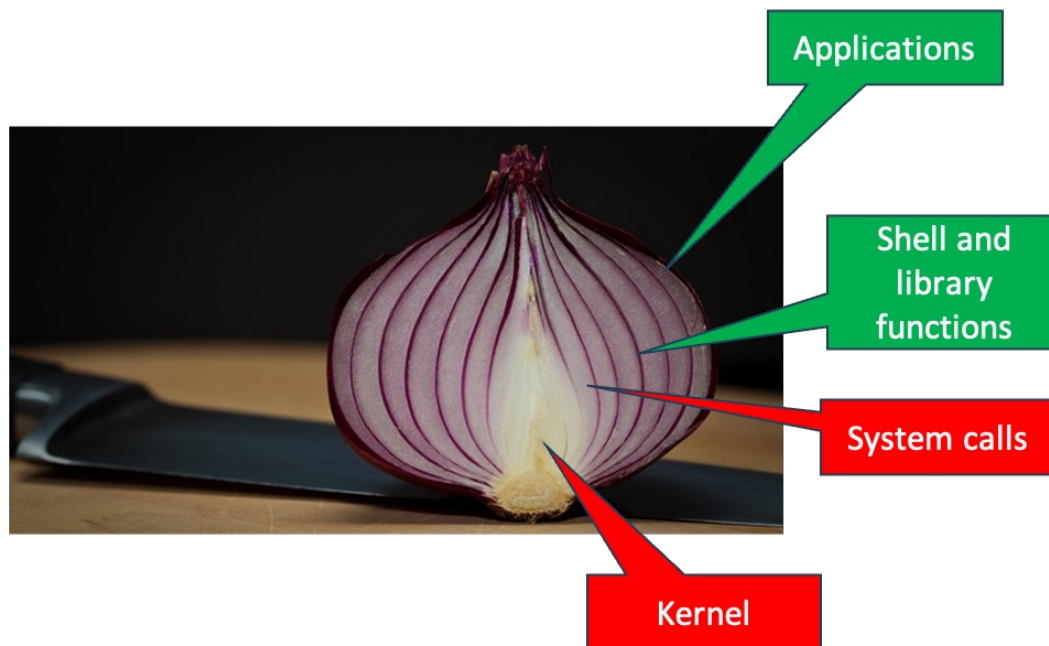
Unix Architecture



Recall the analogy on Slide #3

- A major goal of OS is to support portability across various architecture
- A layered approach helps in achieving this goal as the innermost layer is only one that has to interact with the hardware
 - This innermost layer is called as OS **kernel** (Our head cook!)
 - It is relatively small, controls the hardware resources, and provides an environment under which programs can run
- System call is the interface to the kernel (e.g., read, open, etc.)
- Library functions are built on top of system calls that applications can use
 - E.g., C-library function malloc uses sbrk system call, open(), etc.
- Shell is a command line interpreter that reads user input and execute commands
 - E.g., bash, csh, etc.

Protection Rings



- Protection rings are hardware supported mechanisms used by the OS for protection of data and functionality
 - E.g., x86 supports four protection levels (0-3)
 - Level-0 is called as **kernel mode** (supervisor mode) and Level-3 is **user mode**
 - Levels 1 and 2 are mostly obsolete due to complexity in managing them
- Unix-like OS execute user applications, shell, and library functions in the user mode whereas kernel/syscalls in the kernel mode

Today's Class

- Unix architecture
- System calls

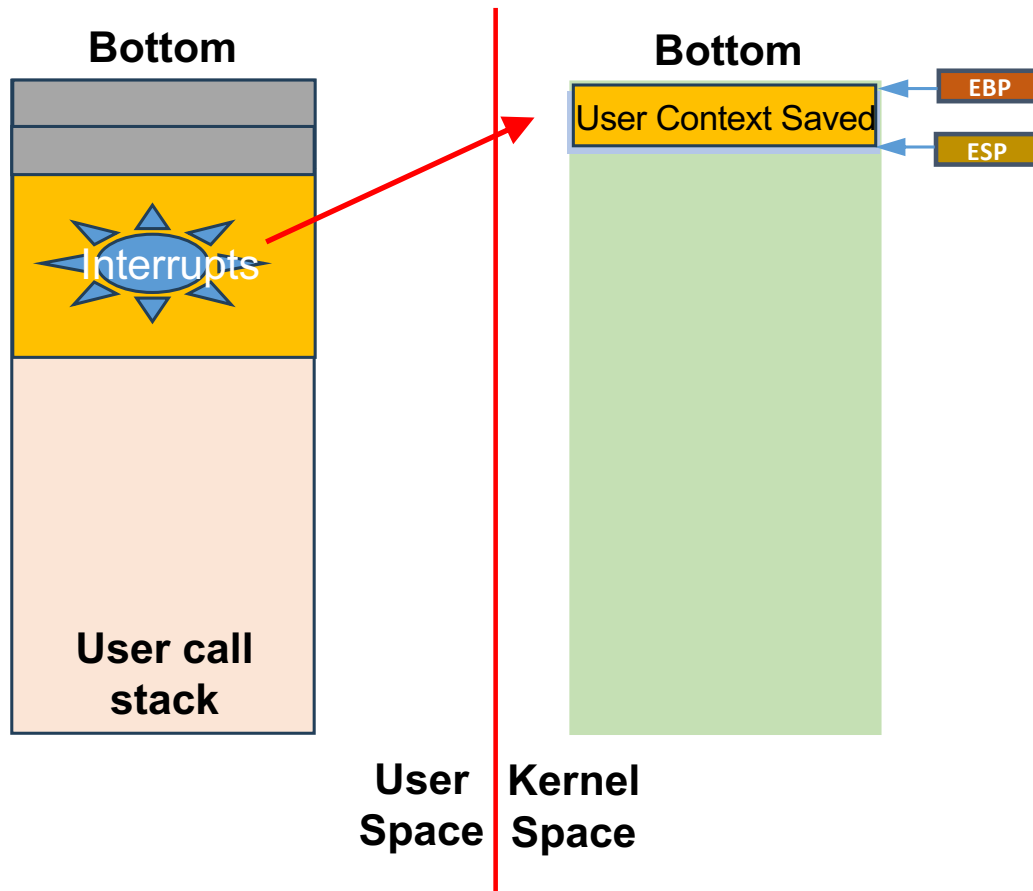
Interrupts: Gateway to the Kernel Mode

- Interrupts are signals to the CPU that something special has to happen
 - INT instruction on x86 to generate an interrupt
 - Interrupts are handled only by the kernel
- Three kinds of interrupts
 - Exceptions
 - Attempt to access invalid memory address, divide by zero, etc.
 - **System calls (software interrupt)** – today's focus
 - Hardware interrupt
 - Signal generated by some hardware device. E.g., disk can generate an interrupt when a block of memory has been read and is ready to be retrieved (e.g., the oven has completed baking pizza and a bell rings to inform the head cook for taking action)
- As per Unix terminology, “interrupts” refer to the hardware interrupts, whereas “trap” refers to software interrupts (e.g., some special type of exceptions and all **system calls**)

libc Syscall Wrapper v/s Direct Syscall

- Code portability across different OS versions
 - Imagine how beneficial is the standard 30s timer **button** in microwave irrespective of the brand and technology used
- Easy to use APIs simplify coding experience as compared to using raw system call APIs
- Error handling support

Two Stacks for Each Processes



- OS maintains two stacks for each processes
 - User stack
 - Kernel stack
- Save user stack context (registers) in kernel mode stack while switching to kernel stack from user stack
 - Restore while jumping back to user stack from kernel stack

Steps for Making a System Call (1/5)

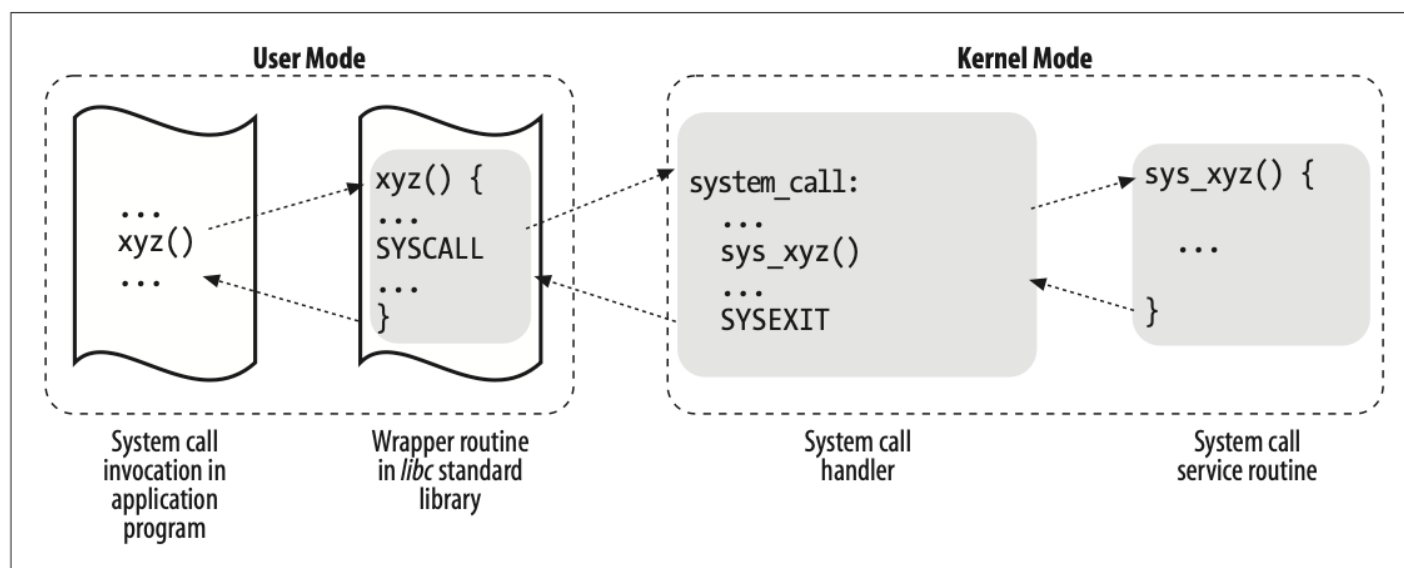
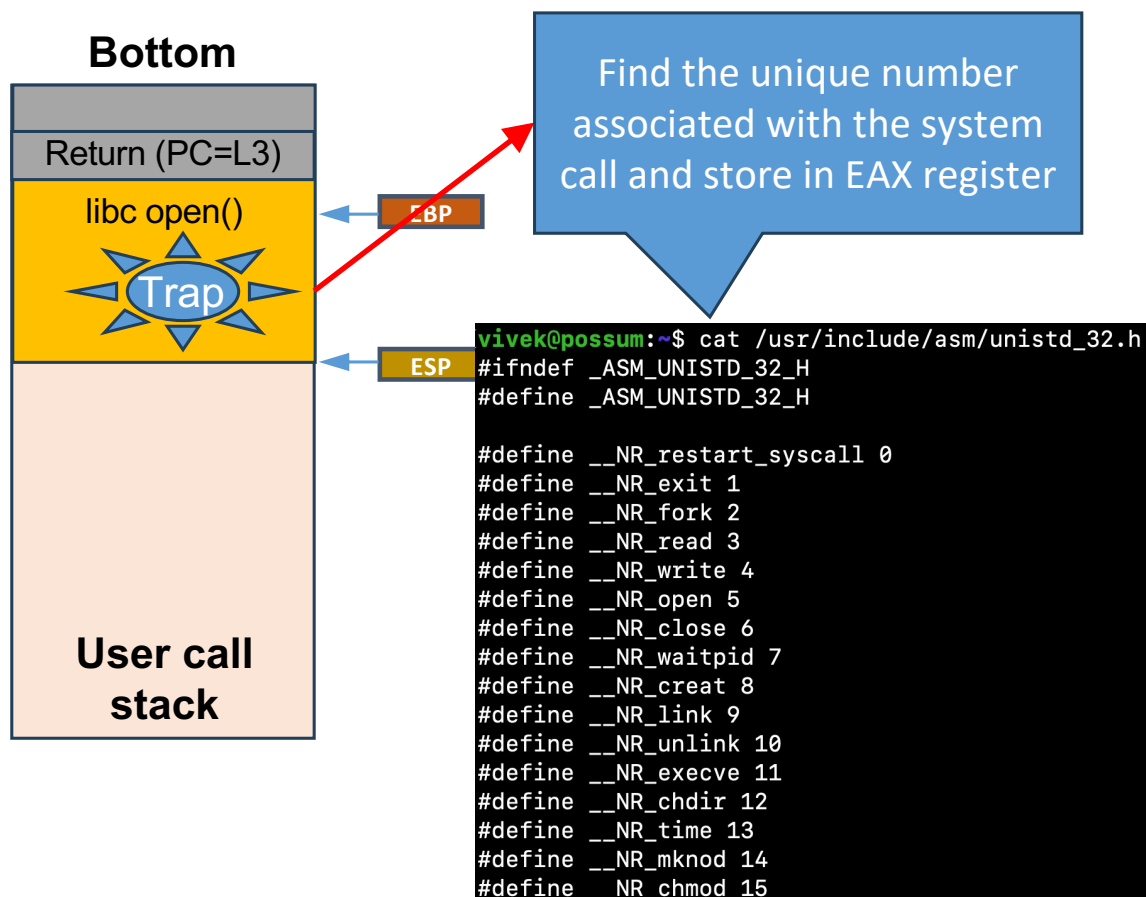


Figure 10-1. Invoking a system call

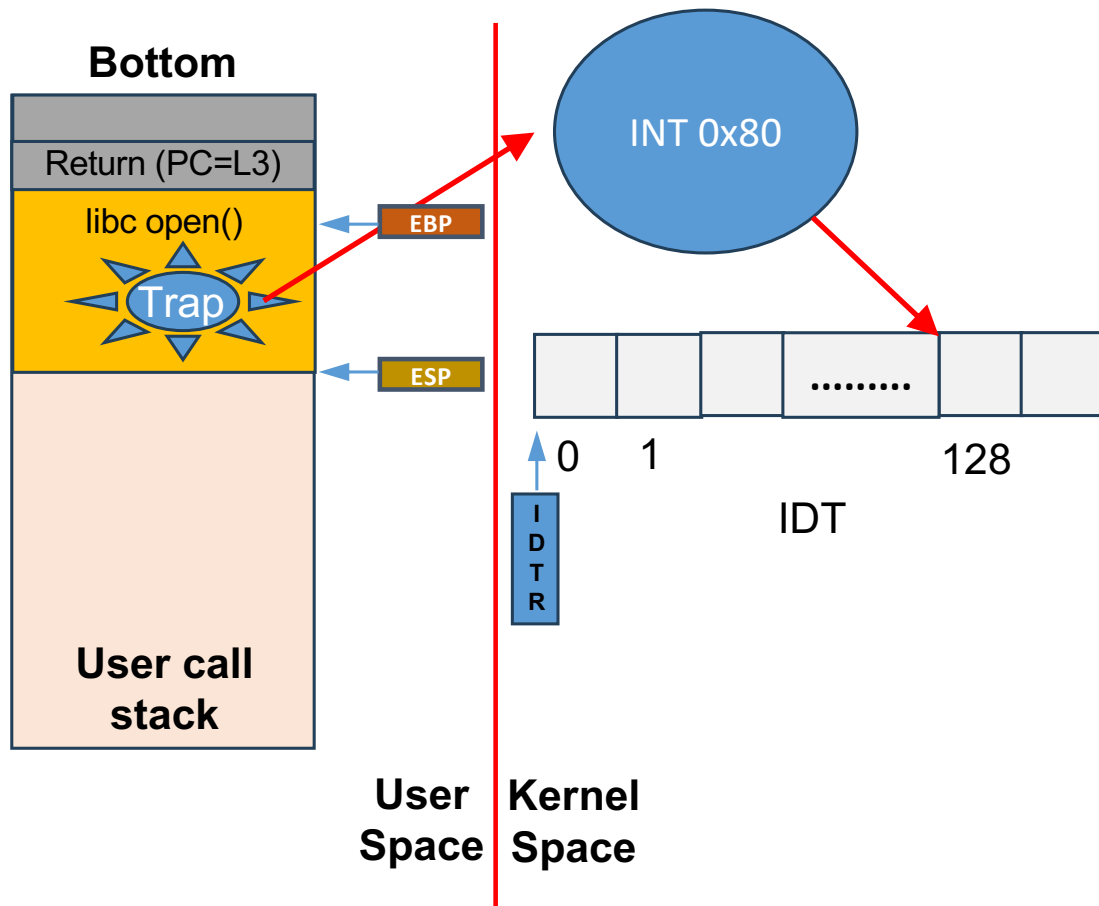
- **At a high level the steps are as follows:**
 1. Save registers in kernel mode stack
 2. Handle the system call by invoking a corresponding system call service routine
 3. Restore the registers and switch back into user mode

Steps for Making a System Call (2/5)



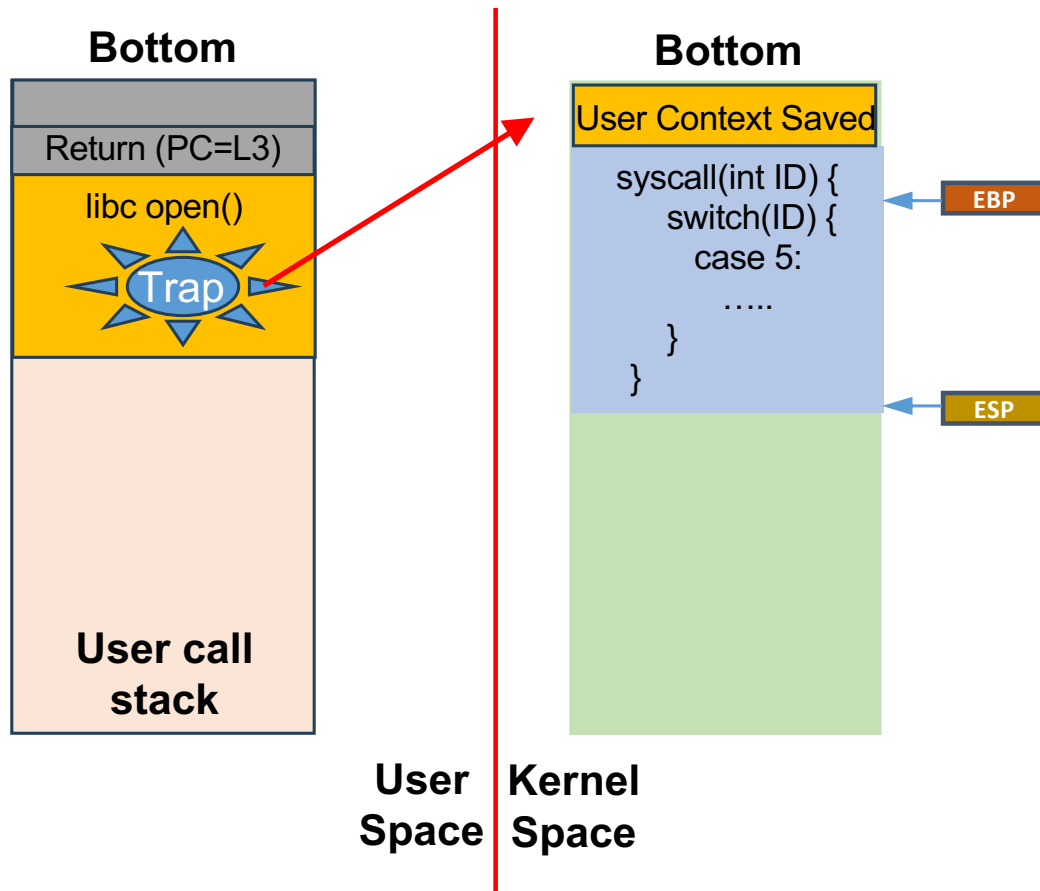
- Each system call has a unique number associated that is declared inside the file `unistd.h`
- Store this syscall number in the EAX register
- Process is still executing inside user space
- Software interrupt (trap) is triggered
 - `int 0x80`

Steps for Making a System Call (3/5)



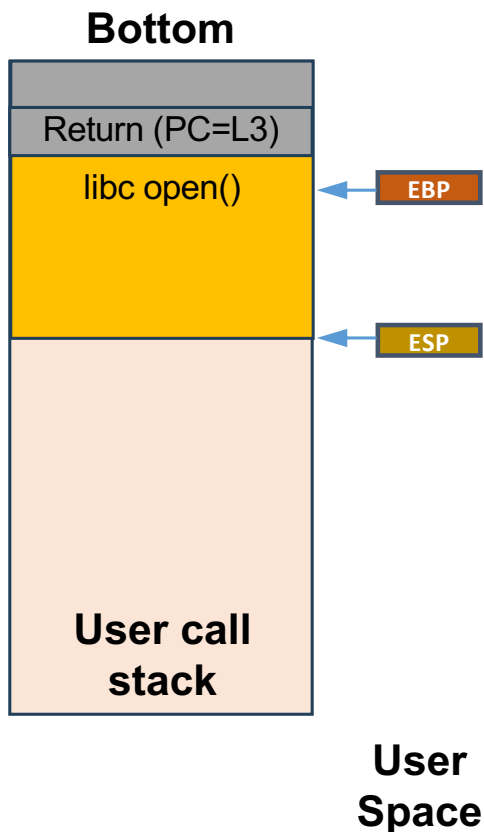
- Kernel sets up Interrupt Descriptor Table (IDT) at boot time whose address is stored in IDT register (IDTR – per CPU). Total 256 entries in IDT on x86
- Each IDT index has the address of some interrupt handler
- 128th entry associated with the system call handler

Steps for Making a System Call (4/5)



- Switch to kernel stack
 - What should be the first step?
- Save registers on kernel stack. These registers are currently holding details to return back to user stack
- Look up the syscall number from EAX and process the syscall

Steps for Making a System Call (5/5)



- For returning to user call stack, first pop the user registers from kernel stack and store them back into respective CPU registers
 - Return back to user call stack in the user mode
 - Protection level-3

Next Lecture

- Process creation and termination